

AI Agent for Lung Disease Detection

By

Muhammad Ehtisham Hanif
(ECI-IT-21-064)

Humair Hayat
(ECI-IT-21-058)

Faizan Rasool
(ECI-IT-21-057)

Under the supervision of

Miss Aisha Shafique

2025

Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan



AI Agent for Lung Disease Detection

By

Muhammad Ehtisham Hanif
(ECI-IT-21-064)

Humair Hayat
(ECI-IT-21-058)

Faizan Rasool
(ECI-IT-21-057)

Under the supervision of

Miss Aisha Shafique

2025

Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan

AI Agent for Lung Disease Detection

By

Muhammad Ehtisham Hanif
(ECI-IT-21-064)

Humair Hayat
(ECI-IT-21-058)

Faizan Rasool
(ECI-IT-21-057)

A project presented to the
Department Of Computing
In partial fulfillment of the
requirements for the degree of
Bachelors of Science
In
Computer Science

Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan



Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan

CERTIFICATE

This project "AI Agent for Lung Disease Detection" presented by **Muhammad Ehtisham Hanif**, **Humair Hayat**, and **Faizan Rasool** under the direction of their project advisor, **Miss Aisha Shafique**, and approved by the project examination committee, has been presented to and accepted by the Faculty of Engineering Sciences and Technology, in partial fulfillment of the requirements for **Bachelor of Science in Computer Science**.

Miss Aisha Shafique (Project Advisor)	Dr. Muhammad Shaheer (Examiner 1)
Sir Usman Javed (Examiner 2)	Dr. Tahir Saleem (Chairman, Department of Computing)

Dr. Hassan Raza
(Associate Dean, FEST)

Abstract

Lung diseases such as tuberculosis (TB), COVID-19 and pneumonia are major causes of illness and death, and in many regions there are not enough radiologists to read chest X-rays quickly. Many student and demo systems train a separate model for each disease, so the models cannot be compared fairly and one image cannot get a single answer covering all diseases. This report presents an **AI Agent for Lung Disease Detection** that sorts chest X-rays into four classes: Normal, Tuberculosis, COVID-19 and Pneumonia. Three models, EfficientNetB3 and DenseNet121 (ImageNet transfer learning) and a custom CNN (trained from scratch), are trained on one shared dataset built from three public sources. Duplicate images are removed, the data is split by patient, class weights handle the small classes, and the transfer models share the same head and fine-tune about 43% of their weights, so the comparison is fair.

A Flask web app checks that each upload is a chest X-ray, runs all three models, and shows each model's probabilities together with a combined soft-voting verdict. On the held-out test set (n = 482), DenseNet121 was the best single model with 90.87% accuracy and 89.14% balanced accuracy, while the ensemble reached 92.32% accuracy and 93.07% balanced accuracy. COVID-19 had high recall but limited precision because it is the smallest class, and a source-leakage probe (88.17% against a 49.79% baseline) showed a moderate source signal. The system is a research demo built on public data and is **not** a clinical diagnostic device.

Acknowledgement

All praises and thanks to Al-Mighty “ALLAH”, the Most Merciful, the Most Gracious, the source of knowledge and wisdom endowed to mankind, who conferred us with the power of mind and capability to take this project to the exciting ocean of knowledge. All respects are for our most beloved Holy Prophet “Hazrat MUHAMMAD (Peace Be Upon Him)”, whose personality will always be a source of guidance for humanity.

Acknowledgement is due to Hamdard Institute of Engineering and Technology for the support of this project, a highly appreciated achievement for us at the undergraduate level.

We wish to express our appreciation to our advisor, Miss Aisha Shafique, for her keen guidance, sincere help, and friendly manner which inspired us to perform well in the project and made it a reality.

Many people, especially our classmates and team members themselves, have made valuable comments and suggestions on this project which gave us the inspiration to improve it further.

Finally, we would like to thank our family and friends for their moral support and encouragement throughout the project. We also acknowledge the providers of the public chest X-ray corpora used in this work, including the Kaggle tuberculosis and pneumonia datasets and the publicly shared COVID-19 radiography collections that made the unified benchmark possible.

Project in Brief

Project Title	AI Agent for Lung Disease Detection
Objective	To build a research chest X-ray system that compares three deep learning models on one shared four-class dataset and makes them available through a web app that combines their results.
Main Contributions	• Fair comparison of three models on 4 classes (same manifest, head and splits) • Flask demo that combines all models and shows when they disagree • Honest evaluation (metrics for each class and a leakage probe)
Limitation	Research demo on small public datasets only; not a clinical diagnostic device.
Undertaken By	• Muhammad Ehtisham Hanif (ECI-IT-21-064) • Humair Hayat (ECI-IT-21-058) • Faizan Rasool (ECI-IT-21-057)
Supervised By	Miss Aisha Shafique
Duration of Project	• Start Date: October 2024 • End Date: September 2025
Tools & Technologies Used	• **Programming Languages:** Python • **Frameworks & Libraries:** TensorFlow, Keras, Flask, OpenCV • **Training pipeline:** `t r a i n i n g /` (manifest, train, evaluate, leakage probe) • **Development Tools:** Jupyter Notebook, Visual Studio Code • **Hardware:** Standard CPU/GPU-based system used for training and testing
System Used	Windows-based development system utilizing CPU/GPU processing for model training and testing.

Contents

Abstract i

Acknowledgement ii

Project in Brief iii

List of Abbreviations ix

1 Introduction 2

1.1 Motivation . . . 2

1.2 Problem Statement . . . 2

1.3 Aim and Objectives . . . 3

1.4 Organization of the Report . . . 3

2 Literature Review 6

2.1 Deep Learning in Medical Imaging . . . 6

2.1.1 Foundational CNN Architectures . . . 6

2.1.2 Large-Scale Chest X-Ray Benchmarks . . . 7

2.2 COVID-19 Detection from Chest X-Rays . . . 8

2.3 Tuberculosis Detection . . . 9

2.4 Pneumonia Detection . . . 10

2.5 Transfer Learning . . . 11

2.6 Data Augmentation and Regularization . . . 11

2.7 Multi-Disease Detection Frameworks . . . 12

2.8 Deployment and Clinical Translation . . . 13

2.9 Explainability and Trust . . . 13

2.10 Gaps in Existing Research . . . 14

2.11 Summary . . . 15

3 Methodology 17

3.1 From Single-Disease Models to a Unified Benchmark . . . 17

3.2 Data Collection and Unified Manifest . . . 17

3.2.1 Source Datasets . . . 18

3.2.2 Rules for a Fair Data Split . . . 18

3.3 Preprocessing and Data Augmentation . . . 19
3.3.1 Model-Specific Preprocessing . . . 19
3.3.2 Augmentation and Class Imbalance . . . 20
3.4 Model Architectures and Shared Head . . . 20
3.5 Training Protocol . . . 21
3.6 Evaluation Methodology . . . 22
3.7 Application Integration Workflow . . . 22
3.8 Methodological Validity and Limitations . . . 22
3.9 Summary . . . 23

4 System Design **25**
4.1 System Architecture . . . 25
4.2 Functional Requirements . . . 26
4.3 Non-Functional Requirements . . . 26
4.4 Tools and Technologies Used . . . 26
4.5 Chest X-Ray Validation Gate . . . 26
4.6 Application Flow and User Interface . . . 27
4.7 Train/Serve Consistency . . . 29
4.8 Summary . . . 30

5 Results and Discussion **32**
5.1 Experimental Setup . . . 32
5.2 Unified Manifest Summary . . . 32
5.3 Four-Class Comparative Results . . . 33
5.4 Per-Class Precision, Recall, and F1 . . . 34
5.4.1 Recall summary . . . 34
5.4.2 DenseNet121 (best single-model macro F1) . . . 34
5.4.3 Ensemble (soft voting) . . . 34
5.5 Confusion Matrices . . . 35
5.6 Leakage Probe . . . 35
5.7 Training Run Summary . . . 36
5.8 Deployed Interface Demonstrations . . . 36
5.9 Challenges Faced and Solutions Applied . . . 38
5.10 Historical Single-Disease Work . . . 38
5.11 Implementation Status . . . 38
5.12 Summary of Findings . . . 39

6 Future Recommendations **41**

6.1 Conclusion 41
6.2 Recommendations 41
6.3 Future Enhancements 41
6.4 Summary of Contributions 41

References **43**

A Dataset Details **46**

A.1 Dataset Overview 46
A.1.1 Data Sources 46
A.1.2 Preprocessing and Splits 46

B Model Architecture Summaries **48**

B.1 Model Training and Implementation 48

C Alignment with Sustainable Development Goals (SDGs) **50**

List of Figures

Figure 3.1 End-to-end methodology pipeline for the unified four-class system 17
Figure 3.2 Dataset unification into a shared four-class training manifest 18
Figure 3.3 Preprocessing and augmentation pipeline with model-specific input sizes and normalisation 19
Figure 3.4 Three architectures scored under a matched four-class protocol (shared head for transfer models) 20
Figure 3.5 Two-phase transfer-learning protocol used for EfficientNetB3 and DenseNet121 21
Figure 3.6 Evaluation flow: matched test scoring, ensemble soft voting, and source-leakage probe 22
Figure 4.1 High-level inference flow for the unified four-class web application . . . 25
Figure 4.2 Three-stage chest X-ray validation gate with optional forced analysis . . . 27
Figure 4.3 Flask upload interface for four-class multi-model chest radiograph analysis 28
Figure 4.4 Chest X-ray validation gate rejection with Analyse anyway option 29
Figure 5.1 DenseNet121 confusion matrix 35
Figure 5.2 Ensemble confusion matrix 35
Figure 5.3 EfficientNetB3 confusion matrix 35
Figure 5.4 Custom CNN confusion matrix 35
Figure 5.5 Consensus Tuberculosis verdict with per-model 4-class probability bars . 36
Figure 5.6 Consensus COVID-19 verdict with per-model 4-class probability bars . . 37
Figure 5.7 Consensus Pneumonia verdict with per-model 4-class probability bars . . 37

List of Tables

Table 2.1 Selected related works versus this project's focus 15
Table 3.1 Source datasets used to construct the unified four-class corpus 18
Table 3.2 Preprocessing per architecture 19
Table 3.3 Shared training hyperparameters and controls 21
Table 4.1 Model integration overview 26
Table 4.2 Flask application components 29
Table 5.1 Training environment (machine that produced the four-class weights) . . . 32
Table 5.2 Selected unified manifest counts by class and split 33
Table 5.3 Class × source contingency (selected manifest) 33
Table 5.4 Overall four-class test metrics ($n = 482$) 33
Table 5.5 Per-class recall on the test set 34
Table 5.6 DenseNet121 per-class metrics 34
Table 5.7 Ensemble per-class metrics 34
Table 5.8 Saved validation metrics and training cost 36
Table 5.9 Challenges and solutions during the 4-class conversion 38
Table 5.10 Implementation status after evaluation export 38
Table A.1 Dataset sources, paths, and known public links 46
Table C.1 Potential educational and research alignment with SDGs (non-clinical) . . 50

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
AUC	Area Under the Curve
BGR	Blue, Green, Red (OpenCV colour order)
BN	Batch Normalisation
CAD	Computer-Aided Diagnosis
CM	Confusion Matrix
CNN	Convolutional Neural Network
COVID-19	Coronavirus Disease 2019
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CXR	Chest X-Ray
DICOM	Digital Imaging and Communications in Medicine
F1	F1-score (harmonic mean of precision and recall)
FR	Functional Requirement
FYP	Final Year Project
GAP	Global Average Pooling
GPU	Graphics Processing Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
HTML	HyperText Markup Language
JPEG / JPG	Joint Photographic Experts Group (image format)
LR	Learning Rate
MD5	Message Digest 5 (hash algorithm)
NFR	Non-Functional Requirement
NIH	National Institutes of Health
P / R	Precision / Recall
PNG	Portable Network Graphics
ReLU	Rectified Linear Unit
RGB	Red, Green, Blue
RT-PCR	Reverse Transcription Polymerase Chain Reaction
SDG	Sustainable Development Goal
TB	Tuberculosis
UI	User Interface
VGG	Visual Geometry Group (network family)
WHO	World Health Organization

Chapter 1

Introduction

1.1 Motivation

Lung diseases are still one of the biggest health problems in the world. According to the World Health Organization, lower respiratory infections are among the top ten causes of death, and tuberculosis alone kills more than 1.5 million people every year [1]. Pneumonia, COVID-19 and other respiratory infections affect people of all ages, but young children (under five) and older people are at the highest risk of serious illness. These diseases also cost a lot in hospital care, lost work and late treatment. The burden is heaviest in low- and middle-income countries, where testing facilities are weakest.

A chest X-ray is usually the cheapest and most easily available scan for checking the lungs. It can be taken quickly, even at the bedside with a portable machine, and it often shows typical signs such as consolidation, cavities or ground-glass opacity. The problem is that someone still has to read the image, and trained radiologists are not available everywhere. In many rural and high-burden areas, X-rays wait a long time before anyone reports on them. This delays triage, isolation and treatment for TB, COVID-19 and pneumonia.

Deep learning has done well on clearly defined chest X-ray screening tasks, especially convolutional neural networks (CNNs) that start from ImageNet weights [2]. A CNN learns useful image features on its own, without hand-made descriptors, and can flag suspicious cases so they are looked at first. Still, many student and demo systems train a separate model for each disease, each with its own labels. With that setup the models cannot be compared fairly, and the user has to guess the disease before uploading an image. Decisions such as which model to deploy, how to check uploads and how to report results for small classes should be based on evidence, not guesswork.

For these reasons, this project uses one shared workflow. There is one set of four labels, three models trained under the same conditions, clear metrics (including recall for each class and a leakage probe), and a single web app that shows the opinion of every model along with a combined verdict. The aim is not to replace radiologists. The aim is a research demo of multi-disease chest X-ray classification that can be repeated, is evaluated honestly, and handles uploads more safely.

1.2 Problem Statement

Many demo systems train separate models for separate diseases, each with different labels. Because of this, the models cannot be compared fairly, and one uploaded image cannot get a single, consistent answer across all diseases. Reported accuracy is also often too optimistic because of data leakage (the same patient or the same file in both training and test data) or because images are processed differently during training and in the app. On top of this, weak input checks can let images that are not X-rays reach the model.

Problem: Design and build a repeatable system that uses several models to classify chest

X-rays as Normal, Tuberculosis, COVID-19 or Pneumonia. All models should use the same data and splits, report clear metrics, and be available through an easy web interface that checks the input and combines the model outputs.

- There are not enough radiologists, and their workload delays the reading of chest X-rays in many places.
- Single-disease models trained on separate datasets cannot be compared, and they split the user's work across different tools.
- A proper research solution has to prevent leakage, process images the same way in training and in the app, and report results for small classes honestly, especially COVID-19, which has very few images.

1.3 Aim and Objectives

Aim:

- To build a research chest X-ray system that compares three deep learning models on one shared four-class dataset and makes them available through a web app that can combine their results.

Objectives:

1. Combine three public chest X-ray datasets into one 4-class list (manifest), remove duplicate images, and split the data by patient.
2. Build EfficientNetB3, DenseNet121 and a custom CNN, train them in the same way, and give the two transfer models the same classification head.
3. Train and test the models using precision, recall and F1 for each class, balanced accuracy, confusion matrices and a source-leakage probe.
4. Build a Flask app that runs every model on each upload, combines them with soft voting, and checks in several stages that the upload is a chest X-ray.
5. Document the method, its limitations and how to repeat it (manifest, seeds and scripts).

1.4 Organization of the Report

Chapter 2 reviews earlier work on deep learning for chest X-rays, transfer learning, multi-disease systems, deployment problems and the research gaps behind this project. **Chapter 3** explains the method: building the shared manifest, making fair splits, training with a shared head, evaluation and how the app is connected. **Chapter 4** covers system design: the software layers, functional and non-functional requirements, the X-ray check, the user interface and keeping training and app processing the same. **Chapter 5** covers the experimental setup, expected performance, comparison tables, challenges, earlier single-disease results (marked as prior work) and the

current state of the implementation. **Chapter 6** gives the conclusion, recommendations, future work and contributions. The appendices give dataset details, model files and the link to the Sustainable Development Goals.

Chapter 2

Literature Review

This chapter looks at published research on deep learning for chest X-rays, mainly for COVID-19, tuberculosis and pneumonia, which are the three diseases this project covers. It goes through the basic CNN architectures, transfer learning, the datasets used, systems that handle more than one disease, and the gap between good lab results and tools that can actually be used. Each section is linked to a decision we made in our system. The architecture studies explain why we chose DenseNet121 and EfficientNetB3 along with a CNN trained from scratch. The transfer learning work explains our two-phase training. The studies on augmentation and regularization guided our shared training settings, and the papers on deployment explain why we built a Flask app that runs several models together (**Chapters 3 and 4**).

2.1 Deep Learning in Medical Imaging

Medical image analysis has moved away from hand-made features and classical machine learning towards end-to-end deep learning. In older pipelines, experts had to decide on the features first (texture descriptors, histogram statistics, edge detectors) and then pass them to a classifier such as a support vector machine or a random forest. A CNN does both steps at once. It learns layers of visual patterns straight from the pixels, so no manual feature design is needed, and on clearly defined screening tasks its results are getting close to those of specialists. This matters a lot for chest X-rays, because disease often shows up as small changes in brightness and texture across the lungs. Fixed hand-made features struggle with these patterns, but convolutional layers can learn them well.

2.1.1 Foundational CNN Architectures

Most chest imaging research builds on a small number of well-known CNN designs. Simonyan and Zisserman [3] introduced the VGG networks and showed that a deep network made of small 3×3 filters extracts strong features. Stacking several 3×3 layers instead of using larger filters adds more non-linearity and a wider view of the image while keeping the number of parameters reasonable. VGG16 and VGG19 are still common baselines for two-class chest X-ray tasks, especially TB screening, where very high accuracy has been reported on public datasets. Their simple structure also makes them easy to fine-tune: you just replace the last fully connected layers with a new classification head.

He et al. [4] proposed ResNet, which uses residual (skip) connections. These solve the degradation problem, where adding more layers to a very deep network actually makes the training error worse. Skip connections let gradients pass straight through identity paths, so networks with hundreds of layers can be trained in a stable way. ResNet versions such as ResNet50 and ResNet50V2 often appear in medical imaging comparisons as reliable general-purpose backbones.

Huang et al. [5] introduced DenseNet. Inside a dense block, each layer receives the feature

maps of all the layers before it, not just the previous one. This helps gradients flow, lets the network reuse features, and needs fewer parameters for the same depth. DenseNet has worked especially well on chest X-rays, where fine texture and opacity details need to survive through the network, and this is why DenseNet121 was adopted for the COVID-19 module in this project.

Tan and Le [6] proposed EfficientNet, which uses compound scaling to grow the depth, width and input resolution of the network together using one fixed coefficient. Instead of making just one of these bigger, it balances all three, which gives good accuracy for the amount of computation used. This makes EfficientNet a good fit when prediction speed and memory matter. For these reasons EfficientNetB3 was selected for the tuberculosis module in this project after the seven-architecture benchmark study.

2.1.2 Large-Scale Chest X-Ray Benchmarks

Deep learning for chest X-rays got a lot of attention after CheXNet by Rajpurkar et al. [7]. They trained DenseNet121 on ChestX-ray14 [8] and showed that it could detect pneumonia about as well as radiologists on a held-out test set. CheXNet gave a sigmoid probability for pneumonia and was trained on more than 100,000 frontal chest X-rays labelled with fourteen chest diseases. After this work, DenseNet became a standard baseline for chest X-ray tasks, and many follow-up studies were done on other lung diseases. The study also showed that, on large medical datasets, deep learning can match specialists when the task is clear and the evaluation is careful.

Irvin et al. [9] released CheXpert, a set of 224,316 chest X-rays labelled for 14 conditions. The labels came from a rule-based labeller, and radiologists reviewed the uncertain cases. Their DenseNet121 baseline reached radiologist-level results on five main conditions, including pneumonia and pleural effusion. It used a loss that handles uncertain labels directly instead of forcing them to be positive or negative. CheXpert and similar large datasets showed that deep learning is more than a research idea and can be clinically useful for chest X-rays, as long as label noise, class imbalance and differences between hospitals are handled with careful preprocessing and validation. The size of CheXpert also showed that training on many diseases at once needs far more data than a student project has, which supported the modular single-task model approach adopted in this report.

Taken together, these benchmark studies point to three ideas that come up again and again. (1) Starting from ImageNet weights is almost always done for chest X-ray tasks, because medical datasets rarely have enough images to train a very deep network from scratch without overfitting. (2) The choice of architecture affects both the best accuracy and the cost of running the model, as the VGG versus EfficientNet trade-off in this project's TB benchmark showed. (3) The size of the dataset and the quality of its labels decide how well a model works on new data, so high accuracy on a public Kaggle dataset does not mean the model will work on images from another hospital or scanner without more testing. All three ideas shaped our method: transfer learning with phased fine-tuning for COVID-19 and TB, a systematic comparison of architectures to

choose the TB model, and testing all three models on fixed held-out splits.

2.2 COVID-19 Detection from Chest X-Rays

The COVID-19 pandemic sped up research on automatic screening from portable chest X-rays. Emergency departments and low-resource hospitals needed fast triage, and RT-PCR testing was often limited or slow. Portable X-ray machines could be used at the bedside in intensive care units and field hospitals and could produce images within minutes. Automatic reading of these images offered a second route alongside lab testing when test kits or lab capacity ran out.

Wang et al. [10] introduced COVID-Net, a custom CNN built for three classes: COVID-19, non-COVID pneumonia and normal. They reported sensitivity above 90%. The study focused on designing the network for the task, for example using depthwise separable convolutions and tailored filter banks, instead of only using ready-made ImageNet classifiers. Their results showed that COVID-19 opacities look a lot like other kinds of pneumonia. This is still a problem for multi-class systems, and it is one reason we use a single shared set of four classes.

Chowdhury et al. [11] published the COVID-19 Radiography Database, one of the public sources merged into our manifest. They showed strong results for COVID-19 versus normal on this data. In our system, these images supply the COVID-19 class (and Normal or Pneumonia labels where they exist). "Viral Pneumonia" images from the COVID sources are put into the shared Pneumonia class, because the Kaggle pneumonia dataset mixes bacterial and viral cases and they cannot be separated again. The images are stored in folders named by class, so they can be loaded by folder when the manifest is built.

Rahman et al. [12] studied how image enhancement and preprocessing affect COVID-19 detection. They found that the right contrast normalisation and enhancement can improve accuracy by several percentage points. This supports using standard preprocessing that matches each model. We follow this in all three models: each uses its own normalisation (`preprocess_input` for DenseNet and EfficientNet, division by 255 for the custom CNN) and is resized to its own expected input size.

Zhang et al. [13] introduced confidence-aware loss functions for COVID-19 detection, which make predictions more reliable when labels are uncertain. Their model reached 96% sensitivity while keeping false positives under control. This matters clinically, because a screening tool should miss as few cases as possible without sending too many people for isolation or referral. During an outbreak, a false negative can let the disease keep spreading, while too many false positives can fill up isolation capacity.

Overall, the COVID-19 literature shows that DenseNet-type networks, fine-tuning in more than one phase, and careful handling of COVID-19 versus other opacities are key to a useful multi-class screening tool. In this project DenseNet121 is one of the three architectures trained on the shared 4-class task, from the same family as CheXNet [7]. Our two-phase training (first a

frozen backbone, then partial unfreezing) follows the step-by-step fine-tuning recommended by Raghu et al. [14]. Making COVID-19 one of four shared output classes, instead of a separate single-disease app, solves the comparison problem described in Section 2.10. These choices appear in the shared training code described in **Chapter 3** and are evaluated in **Chapter 5**.

2.3 Tuberculosis Detection

Tuberculosis is still one of the leading infectious causes of death in the world. The World Health Organization [15] reports more than 1.5 million TB deaths every year, and late diagnosis keeps the disease spreading, especially in high-burden areas with few radiologists. Chest X-rays play a central part in TB screening. Signs such as upper-lobe infiltrates, cavities and pleural effusion can point to active disease even before lab confirmation is available. Automatic TB detection from frontal chest X-rays is therefore a clear screening task with real public health value.

Jaeger et al. [16] set up the first standard TB detection benchmarks using the public Montgomery County and Shenzhen Hospital datasets. These came with standard evaluation splits, so different algorithms could be compared fairly. Before these datasets were released, TB studies used their own private data, which made it hard to compare results. The Montgomery and Shenzhen datasets are still reference points for both classical and deep learning TB research, and they influenced the later Kaggle TB datasets used in this project [17].

Lakhani and Sundaram [18] used an ensemble of GoogLeNet and AlexNet [19] on chest X-ray datasets and reached AUC values of 0.99 and 0.97 respectively. This was among the first studies to show that deep learning could match or beat radiologists at classifying TB from frontal chest X-rays. Their ensemble combined two architectures that look at the image in different ways, which is similar in spirit to the multi-architecture comparison done for the TB module in this project.

Hwang et al. [20] showed that a CNN trained on more than 40,000 chest X-rays reached an AUC of 0.96 for TB screening. This showed the value of large-scale training even before ImageNet transfer learning became common in medical imaging. Their work suggests that, with enough labelled examples, a large dataset can make up for a simpler architecture.

More recent studies have compared modern architectures on Kaggle TB datasets. They usually find that fine-tuned VGG models reach very high accuracy on the two-class Normal versus Tuberculosis task, while EfficientNet models give similar accuracy at a lower computing cost. Tan and Le [6] showed that compound scaling lets EfficientNetB3 come close to the accuracy of heavier networks while costing less to run. This trade-off is why we included EfficientNetB3 in our 4-class comparison, together with DenseNet121 and a custom CNN.

From the TB literature we take two points. First, architectures need to be compared properly, because the best accuracy on a single two-class task does not carry over automatically to a harder shared multi-class problem. Second, two-class TB screening on cleaned public datasets can reach

near-perfect accuracy when transfer learning and regularization are used correctly, but those numbers cannot be compared with 4-class, patient-grouped results. Real-world performance still needs solid preprocessing, testing on held-out data and clinical validation.

2.4 Pneumonia Detection

CNN-based pneumonia detection has been tested many times on the Kaggle Chest X-Ray dataset shared by Mooney [21]. It contains chest X-rays of children labelled NORMAL or PNEUMONIA, about 5,863 images split into training, validation and test folders. With a dataset of this size, overfitting is a known risk, so regularization is needed. Because the train, validation and test folders are fixed in advance, researchers can report results on the same test set and compare architectures under the same conditions.

Studies on this dataset regularly report accuracy above 90%, using everything from simple sequential CNNs to complex transfer learning models. Rajpurkar et al. [7] and later CheXpert-related work [9] treat pneumonia as one of the most studied chest X-ray findings, with CNNs learning the opacity, consolidation and interstitial changes caused by bacterial and viral infection. On frontal X-rays of children, pneumonia usually appears as increased opacity in one or both lungs, often with air bronchograms visible inside the consolidated areas. Convolutional filters can learn these patterns when there are enough labelled examples.

Batch normalization, dropout and learning rate scheduling are repeatedly shown to help models generalise on a dataset of this size. Batch normalization keeps the activations stable across mini-batches. Dropout switches off random neurons during training so they do not depend too much on each other. Reducing the learning rate when progress stalls allows smaller, finer weight updates once the validation loss stops improving. Shallow to medium-depth custom CNNs are still competitive when training time, model size and an easy-to-understand architecture matter more than a small accuracy gain from a very deep transfer learning backbone. This is why we designed the five-block custom CNN used as Model 3.

Rahman et al. [12] also looked at preprocessing for pneumonia and COVID-19 classification. They noted that pixel normalisation, the resize method and the colour channel order all affect predictions, especially when images are loaded with OpenCV and converted between BGR and RGB. Our shared preprocessing handles this the same way for all models. If preprocessing during training and during prediction does not match, performance can drop without any obvious error, which is a common problem in deployed medical AI systems.

The Kaggle pneumonia dataset only labels images as NORMAL or PNEUMONIA and does not give the type or cause of pneumonia. We map these folders to the shared Normal and Pneumonia classes of our 4-class label set, together with the TB and COVID sources, so that all models are compared using the same output classes.

2.5 Transfer Learning

Transfer learning from models pre-trained on ImageNet [2], including VGG16 [3], InceptionV3, ResNet50 [4], DenseNet121 [5] and EfficientNet [6], has become the standard approach in medical imaging AI when labelled medical data is limited. ImageNet has over a million natural images in a thousand categories. Training on it teaches a model general low-level features such as edges, textures and shapes, and these still work on X-rays even though X-rays look very different from normal photos. Medical datasets rarely have enough images to train a very deep network from random weights without heavy overfitting.

Raghu et al. [14] studied how well transfer learning works for medical imaging in their Transfusion study. They found that low-level features transfer well, but higher-level features need fine-tuning for the new task. They also saw that freezing the early layers and then gradually unfreezing deeper blocks (or first training a new head before full fine-tuning) often works better than training from random weights on small medical datasets. Keeping the backbone frozen at the start stops large gradient updates from damaging the pre-trained features before the new head has learned anything useful.

This finding directly shaped the two-phase training of the DenseNet121 COVID-19 model. In Phase 1 the backbone is frozen and only the new head is trained, with a learning rate of 10^{-4}. In Phase 2 the upper backbone layers from index 300 onwards are unfrozen and trained with a lower learning rate of 10^{-5} to adapt them to X-rays. The EfficientNetB3 TB model follows the same idea: the ImageNet backbone starts frozen, and the head is trained with in-graph augmentation before the weights are saved for deployment.

The literature also warns that transfer learning does not guarantee good results on new data. On small public datasets, performance can look better than it really is because of train-test leakage, repeated tuning on the validation set, or dataset-specific artifacts such as hospital borders or text on the image that the model learns to use as a shortcut. Fixed random seeds, a held-out test set, and comparing architectures on the same splits (as in the TB benchmark with seed 1337) are the recommended practices from this work.

2.6 Data Augmentation and Regularization

Medical imaging datasets are usually small compared with natural image datasets, so augmentation and regularization are very important for avoiding overfitting. Augmentation makes the training set effectively bigger by applying changes that do not change the label, which pushes the model to learn general features instead of memorising exact pixels. The literature points to a few augmentations that work well for chest X-rays: random rotation and horizontal flips (since frontal X-rays are roughly symmetric left to right), width and height shifts to imitate small changes in patient position, zoom changes to imitate different image scales, and other geometric changes that keep the intensity the same. These are applied during training only.

Dropout, batch normalization, early stopping, reducing the learning rate on a plateau, and model checkpointing show up again and again in the best lung classification studies. Early stopping watches the validation loss and stops training if it does not improve for a set number of epochs, so the model does not keep fitting noise. Checkpointing saves the best weights seen during training instead of the weights from the last epoch, which may already be overfitted. Irvin et al. [9] and Chowdhury et al. [11] both use augmentation together with regularization when fine-tuning deep networks on unbalanced disease labels.

For both two-class and multi-class models (sigmoid or softmax outputs), class imbalance is common in pneumonia datasets, where positive cases can make up most of some splits. It is often handled by augmenting the smaller class, splitting the data carefully by class, and using early stopping, rather than simply chasing accuracy. Accuracy can be misleading when classes are unbalanced: a model that always predicts the largest class can show high accuracy while getting every minority case wrong. We use these techniques in the same way for all three models, with augmentation settings adjusted to each model's input size and design. For example, DenseNet121 uses `ImageDataGenerator` with rotation and shift settings, while EfficientNetB3 puts `RandomZoom` and `RandomRotation` directly inside the Keras model.

2.7 Multi-Disease Detection Frameworks

Systems that detect several diseases have received much less attention than single-disease work. There are many studies on individual diseases (COVID-19 detection, TB screening, pneumonia classification), and each reports good accuracy on its own. But most of these tools cover only one disease and report offline numbers without being part of a single clinical workflow. In practice, a doctor who suspects more than one possible cause would need to run several separate apps, each with its own interface, preprocessing and output format. This is not practical in emergency triage, where speed and simplicity matter most.

CheXpert [9] and CheXNet [7] show multi-label prediction from one model trained on many diseases at once. However, this needs very large labelled datasets, and it gives all predictions together instead of letting the user screen for a specific suspected condition. In real triage, doctors often work through a list of possible diagnoses: they might suspect TB where it is common, COVID-19 during an outbreak, or community-acquired pneumonia in a child.

Our framework fills this gap by training three architectures (EfficientNetB3, DenseNet121 and a custom CNN) on one shared four-class dataset (Normal, Tuberculosis, COVID-19, Pneumonia). This makes the comparison fair, and every model can name the same diseases. Each backbone keeps its own input size and preprocessing (DenseNet 224×224, EfficientNet 256×256, custom CNN 150×150), but they all use the same splits, seed, augmentation budget, the same head design for the transfer models, and a softmax output over four classes. The Flask app runs every available model on each upload and combines their probabilities with soft voting. It shows where

the models agree and disagree, instead of making the user pick one disease model first. This kind of modular, multi-architecture design with one shared label set is not common in published medical AI work, which usually reports single-disease notebooks or trains one large multi-label network on datasets like CheXpert without a controlled comparison of backbones.

2.8 Deployment and Clinical Translation

A common criticism of medical AI research is the "deployment gap". Many studies report accuracy, sensitivity and AUC on past datasets but never deliver software that doctors can use. Abadi et al. [22] and the TensorFlow tools that followed made it practical to save models and run them at scale, which made deployment easier. Even so, published chest X-ray studies often stop at a notebook. They report test-set numbers but do not show that a non-expert can reliably use the model through a stable interface. This matters, because a model with 95% accuracy in a controlled experiment is of no practical use if it cannot be loaded, fed the right input and run reliably in a hospital or community setting.

A usable system needs more work than just training the model. It needs input checks in several stages to reject uploads that are not X-rays. It needs preprocessing at prediction time that exactly matches training (including bilinear resizing instead of the default nearest-neighbour). It needs lazy loading of models so the app starts quickly even with several large weight files, and safe handling of requests that arrive at the same time. Bradski [23] established OpenCV as a standard library for reading and checking images. We use it in the three-stage chest X-ray check (monochrome check, radiograph envelope features and anatomy check) that runs before prediction.

Lightweight Flask-based [24] web apps have become a common, easy way to deploy medical AI in student and research prototypes. In this project, the Flask app has an upload page that takes an image file, checks it, runs `predict_all()` on every trained 4-class model, and shows probability bars for each model together with a soft-voting ensemble, without asking the user to choose a disease model. Because prediction runs on the server, the model weights and TensorFlow stay on the server. Clinical deployment at a regulatory level is a separate matter from research prototypes, but a working interface is still needed for a project to be credible, even for an academic final year project.

This project tackles the deployment gap directly with a single Flask web app that runs all three 4-class models and presents their combined result, which answers the gap described in Section 2.10.

2.9 Explainability and Trust

Doctors will only use AI screening tools if they can understand and trust them, not just if the tools are accurate. A doctor who gets a prediction without knowing what it is based on is unlikely

to rely on it for triage, especially when it does not match their own judgement. Selvaraju et al. [25] introduced Grad-CAM (Gradient-weighted Class Activation Mapping), which creates heatmaps showing the parts of an image that most affected a CNN's prediction. Grad-CAM takes the gradient of the target class score with respect to the feature maps of a chosen convolutional layer and produces a rough map that can be laid over the X-ray.

Grad-CAM and similar methods have been used on chest X-ray models to check that they look at sensible areas, such as the lungs, rather than borders, text or other dataset artifacts. If a model highlights a hospital name printed on the edge of the image instead of the lung opacities, the heatmap exposes a false shortcut that accuracy numbers alone would hide. Even with all this interest, explainability is rarely built into multi-model screening systems with one interface. Most explainability studies look at a single model and a single disease. Adding Grad-CAM or a similar visual explanation to a multi-disease system is still open future work, as noted in the research gaps below and in **Chapter 6**.

2.10 Gaps in Existing Research

Deep learning for chest X-rays has made a lot of progress, but turning research results into multi-disease screening tools that doctors can use is still unfinished. Many studies report strong accuracy on cleaned public datasets, but few deliver complete software that a doctor could actually use during triage. The gaps below are not just theory. They directly shaped the design decisions, architecture choices and deployment work described in **Chapters 3 to 5**. Closing them needs better models, but also attention to software integration, careful validation and user-facing deployment, which get much less attention in published work than new architectures do.

1. **Single-disease models that cannot be compared:** Separate student or demo classifiers with different label sets make fair comparison impossible and split the user's work across several tools.
2. **Splits that ignore leakage:** The same patient or the same file in both training and test data makes the reported accuracy look better than it is.
3. **Reporting only accuracy:** A single accuracy number hides poor recall on small classes (especially COVID-19, which has few images).
4. **Training and app mismatch:** Resizing or normalising images differently in the app than in training quietly lowers the quality of live predictions.
5. **Weak input checks:** Checking only whether an image is monochrome still lets many non-X-ray images through.
6. **No visible model disagreement:** Few demos show where models disagree alongside the combined result.

2.11 Summary

Past research supports using CNNs and ImageNet transfer learning for chest X-rays, but it also warns that small COVID datasets, dataset shortcuts and reporting only accuracy make results hard to trust. Careful data handling (patient-grouped splits, removing duplicate images), the same training setup for a fair comparison of backbones, and usable interfaces that clearly state they are not for diagnosis are all essential. This project responds with a shared four-class comparison of several architectures, evaluation that checks for leakage (including a source probe), and a Flask ensemble app with a tested X-ray check. Grad-CAM-style explanations are left for future work.

Table 2.1: Selected related works versus this project's focus

Work	**Task**	**Dataset style**	**Method**	**Limitation**
CheXNet [7]	Multi-label CXR	Large NIH corpus	DenseNet121	Multi-label setup, not four separate classes; needs a very large dataset
COVID-Net [10]	COVID multi-class	Custom CXR set	Custom CNN	Only one model; data collected in a rush during the pandemic
Kaggle pneumonia CNNs [21]	Binary pneumonia	Mooney/RSNA-style	Shallow / transfer CNNs	Only two classes; train and test images may overlap
Binary TB transfer studies [18, 20]	TB vs Normal	Public TB CXR	VGG / EfficientNet	Only TB vs Normal; results cannot be compared with other diseases
This project	4-class single-label	Merged public sources	3 architectures + ensemble	Small public data; research demo, not for clinical use

Chapter 3

Methodology

This chapter explains how the system was built and tested. The goal was not only to classify lung diseases from chest X-rays, but also to compare three deep learning models fairly, with all of them trained under the same conditions so the results can be repeated. The work has six steps: combining the datasets, preprocessing and augmentation, designing the models, training, evaluation (including a leakage check), and deploying the models in a Flask app that combines their results.

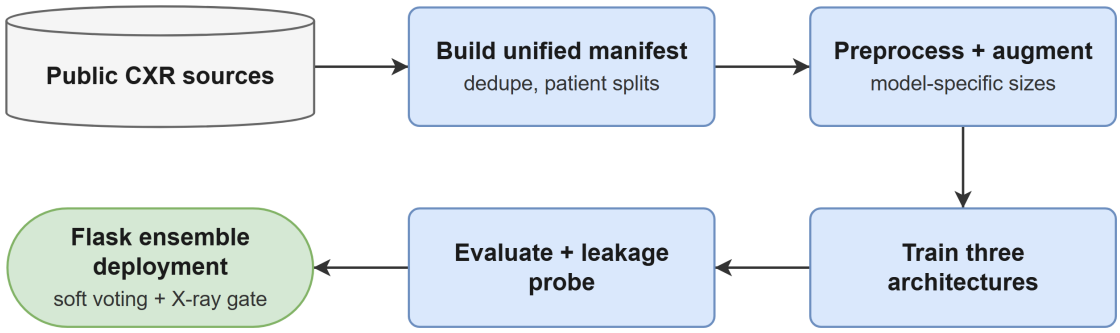


Figure 3.1: End-to-end methodology pipeline for the unified four-class system

3.1 From Single-Disease Models to a Unified Benchmark

In an earlier version of this project we had three separate models, each with its own labels: a two-class tuberculosis model, a multi-class COVID-19 model (which used to include Viral Pneumonia), and a two-class pneumonia model. These models could not be compared properly, because any difference in results might come from the datasets and not from the models themselves. In the new method, the task becomes a single four-class problem. All three architectures are trained on the same merged dataset, with the same patient-grouped splits, the same classification head design (for the transfer models) and the same training budget. The label order is fixed as Normal, Tuberculosis, COVID-19, Pneumonia. Viral Pneumonia images from the COVID sources are added to Pneumonia, because the Kaggle pneumonia dataset mixes bacterial and viral cases and they cannot be separated afterwards.

3.2 Data Collection and Unified Manifest

We did not collect any new private clinical data. A script (`training/build_manifest.py`) combines three public datasets into one file, `training/manifest.csv`. For every image this file stores the path, the class label, the patient ID, the source dataset and whether it belongs to the training, validation or test set.

3.2.1 Source Datasets

Table 3.1: Source datasets used to construct the unified four-class corpus

Source dataset	Expected path	Classes contributed
TB Chest X-Ray (Kaggle) [17]	`TB_Chest_Radiography_Database/`	Tuberculosis, Normal
Covid19-dataset	`DenseNet121/Covid19-dataset/`	COVID-19, Normal, Pneumonia
Chest X-Ray Pneumonia (Kaggle) [21]	`pneumonia_detection/data/chest_xray/`	Pneumonia, Normal

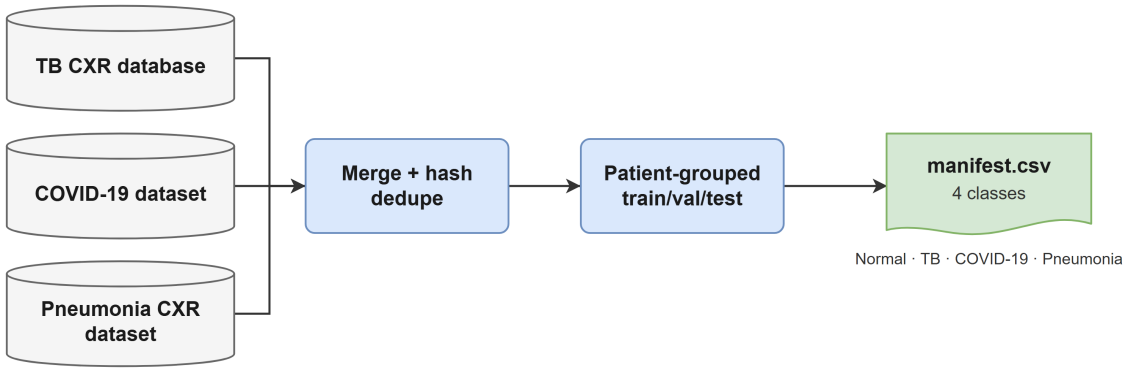


Figure 3.2: Dataset unification into a shared four-class training manifest

3.2.2 Rules for a Fair Data Split

1. **Remove duplicate images.** If the exact same image appears twice, only one copy is kept. This stops the same image from being in both the training and the test set.
2. **Keep each patient in one set.** All images of one patient go into only one set (training, validation, or test), never into two.
3. **Give every patient a unique ID.** Different COVID folders reuse the same file names (for example `0101.jpeg`), so the folder path is added to each ID to keep patients apart.
4. **Take Normal and Pneumonia from more than one dataset.** This stops a model from guessing the class just by recognising which dataset an image came from.
5. **Limit large classes and use class weights.** COVID-19 has only 130 images after removing duplicates. The other classes were limited to 1,200 images each (`--cap 1200`). Class weights give the smaller classes more importance during training (COVID weight ≈ 2.851).
6. **Check classes against sources.** Training starts only after confirming that Normal and Pneumonia images come from more than one dataset.

All images come from public research datasets. The system is a research demo and a comparison of models. It is not a clinically tested diagnostic device.

3.3 Preprocessing and Data Augmentation

3.3.1 Model-Specific Preprocessing

Table 3.2: Preprocessing per architecture

Model	Input size	Preprocessing
EfficientNetB3	$256 \times 256 \times 3$	EfficientNet `preprocess_input`
DenseNet121	$224 \times 224 \times 3$	DenseNet `preprocess_input`
Custom CNN	$150 \times 150 \times 3$	Divide by 255

Training and prediction use exactly the same steps to read and resize images (bilinear resizing). Earlier, the app resized images with nearest-neighbour while training used bilinear. This changed a large share of the pixel values (a training and app mismatch), so it was fixed.

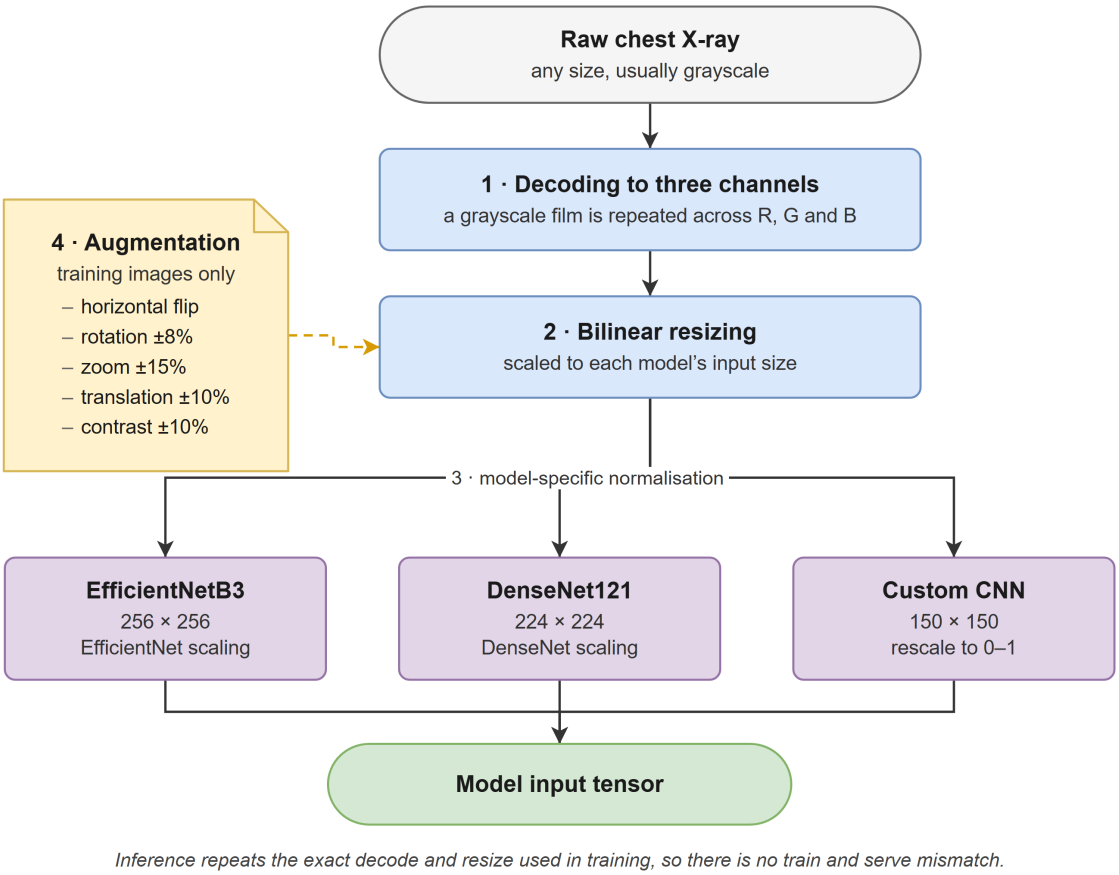


Figure 3.3: Preprocessing and augmentation pipeline with model-specific input sizes and normalisation

3.3.2 Augmentation and Class Imbalance

All models get the same geometric augmentation (rotation, shifting, zoom and horizontal flip, each within set limits) through the shared data pipeline. We did not oversample the disease images. Instead, class weights based on how rare each class is are used with categorical cross-entropy, so that mistakes on the smaller classes count for more.

3.4 Model Architectures and Shared Head

The models are defined in `training/models.py`. Both transfer models use the same head, so that the comparison measures the backbone and nothing else:

$$\text{GAP} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Dense}(256, \text{ReLU}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(4, \text{softmax}).$$

Every model uses a softmax output with categorical cross-entropy. This fixes an earlier mistake where a sigmoid output was paired with a categorical loss. None of the models start from the old single-disease weights.

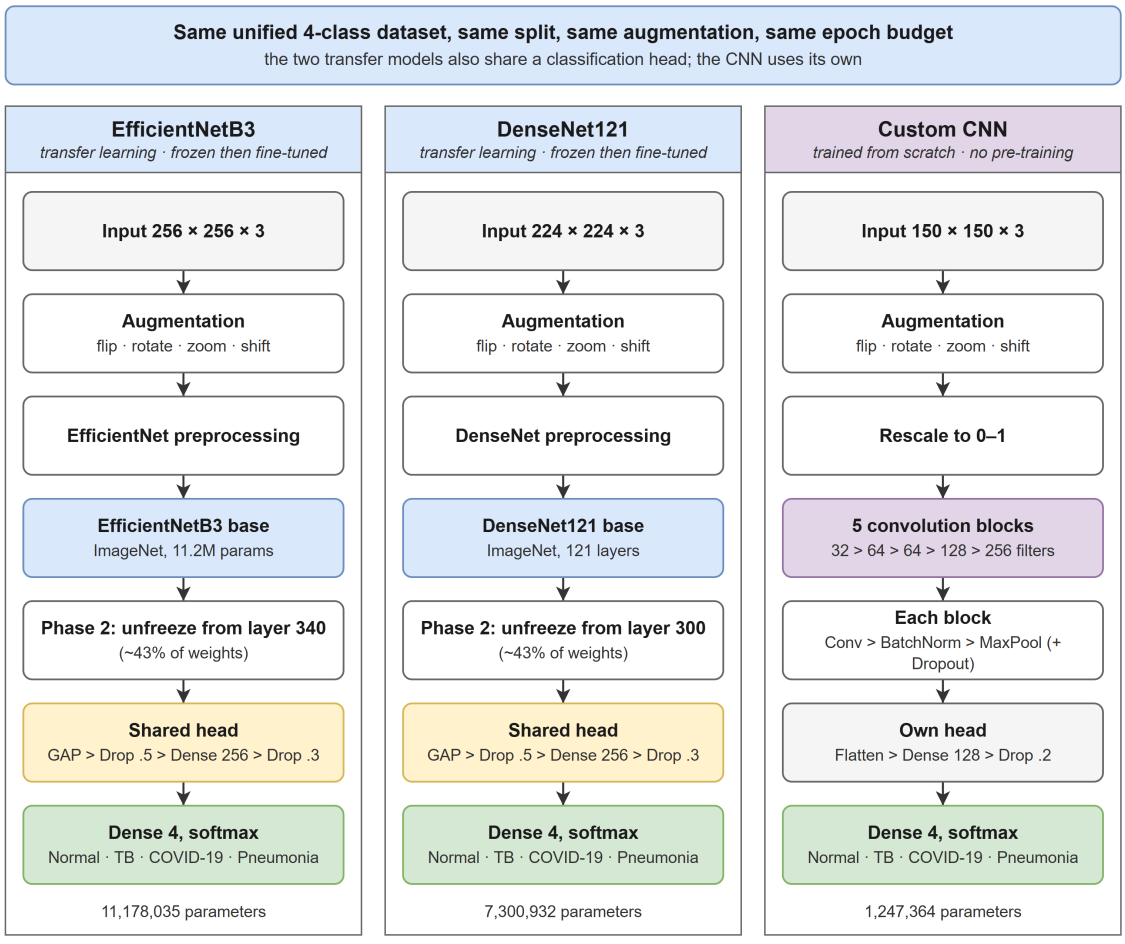


Figure 3.4: Three architectures scored under a matched four-class protocol (shared head for transfer models)

EfficientNetB3. Uses an ImageNet backbone. In Phase 2, layers from 340 onwards are unfrozen (~42.7% of the parameters).

DenseNet121. Uses an ImageNet backbone. In Phase 2, layers from 300 onwards are unfrozen (~42.8%). The BatchNorm layers stay frozen during fine-tuning.

Custom CNN. Five convolution blocks (filters 32 → 64 → 64 → 128 → 256) with BatchNorm, pooling and dropout, ending in a four-class softmax. It is trained from scratch and serves as a baseline.

3.5 Training Protocol

Table 3.3: Shared training hyperparameters and controls

Setting	Value
Shared seed	1337
Batch size	32 (identical across models when resources allow)
Phase-1 learning rate	1×10^{-4} (Adam), backbone frozen
Phase-2 learning rate	1×10^{-5} (Adam), partial unfreeze for transfer models
Loss	Categorical cross-entropy with class weights
Output	4-way softmax
Test set	Never used for training or model selection
Artefacts	`weights/*_4class.keras`, `training/runs/*.json`

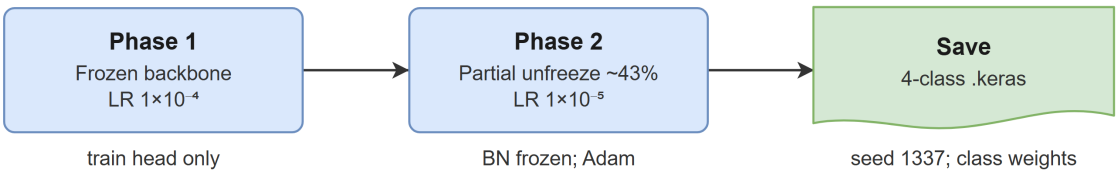


Figure 3.5: Two-phase transfer-learning protocol used for EfficientNetB3 and DenseNet121

During training we use checkpointing, early stopping and ReduceLROnPlateau. All models can be trained with `python training/train.py --model all`.

3.6 Evaluation Methodology

The script `training/evaluate.py` tests every model on the held-out test set. It reports accuracy, balanced accuracy (the average recall over the classes), precision, recall and F1 for each class, confusion matrices, a comparison between the models, and the results of the soft-voting ensemble. These are saved to `training/results/comparison.md`. Soft voting takes the average of the models' probability outputs and picks the class with the highest value. Unlike a simple majority vote, it keeps information about how confident each model is.

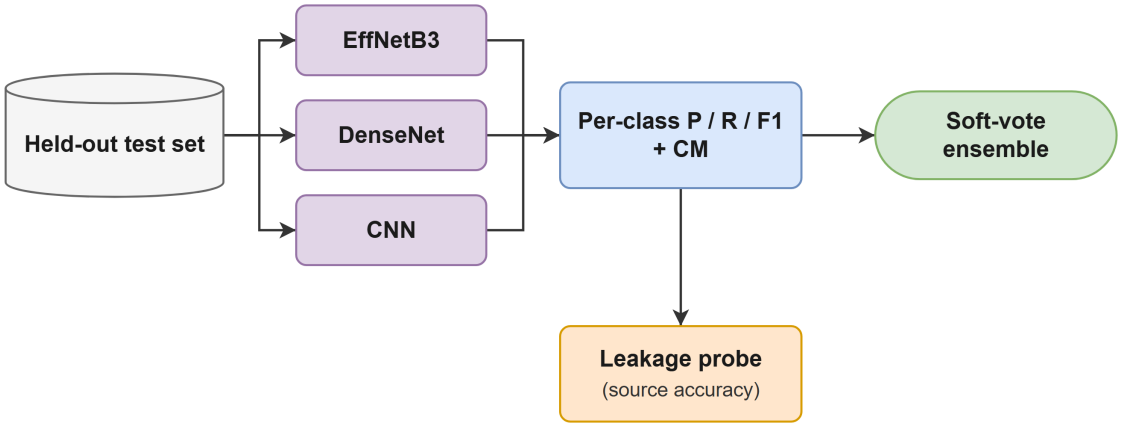


Figure 3.6: Evaluation flow: matched test scoring, ensemble soft voting, and source-leakage probe

The script `training/leakage_probe.py` trains a classifier that tries to guess which dataset an image came from, and compares it with a simple majority baseline. A result close to the baseline is what we want. If the source could be predicted almost perfectly, it would be a warning that TB or COVID-19 might be recognised from how the images were taken rather than from the disease. COVID-19 recall is still the main weak point, because there are so few COVID-19 test images.

3.7 Application Integration Workflow

In the app, the flow is: upload (PNG/JPG/JPEG, up to 16 MB) → three-stage chest X-ray check (or "Analyse anyway") → `predict_all()` → soft-voting ensemble → result page with a probability bar for each model, the final verdict and whether the models agree. The model weights must have four outputs, and old `.h5` files are rejected. The full sequence for running the project is: `build_manifest.py` → `train.py --model all` → `evaluate.py` → (optional) `leakage_probe.py` → `Frontend/app.py`.

3.8 Methodological Validity and Limitations

The method is sound because the task is clearly defined, there are controls against leakage, the models are trained under the same conditions, the metrics include recall for each class, and

the app processes images the same way as training. It also has known limitations. The public datasets are small. COVID recall is not statistically stable. Four-class accuracy is expected to be lower than the older two-class figures. TB and COVID images each come from a single source, so a model could still take shortcuts (which is why we run the probe). The system is not a clinical device, and Grad-CAM and testing on data from several hospitals are left for future work.

3.9 Summary

In short, this method turns three previously separate detectors into one four-class comparison of three models that guards against leakage and is deployed as an ensemble. The comparison is fair because all models share the same data and training setup, and the evaluation is open about the limits of the datasets and the fact that this is not a diagnostic tool.

Chapter 4

System Design

This chapter shows how the method from **Chapter 3** was turned into working software. The system has five layers: the user interface (HTML/CSS/Jinja), the application (Flask routes), the prediction service (lazy loading, locking and the ensemble), the saved models (`weights/`), and an offline training pipeline in `training/`.

4.1 System Architecture

When a user uploads an image, a Flask route receives it and the chest X-ray check makes sure it is a valid X-ray. Then `predict_all()` in `predictors.py` runs EfficientNetB3, DenseNet121 and the Custom CNN on it. Their outputs are combined with soft voting, and the result page shows the probability bars and whether the models agree.

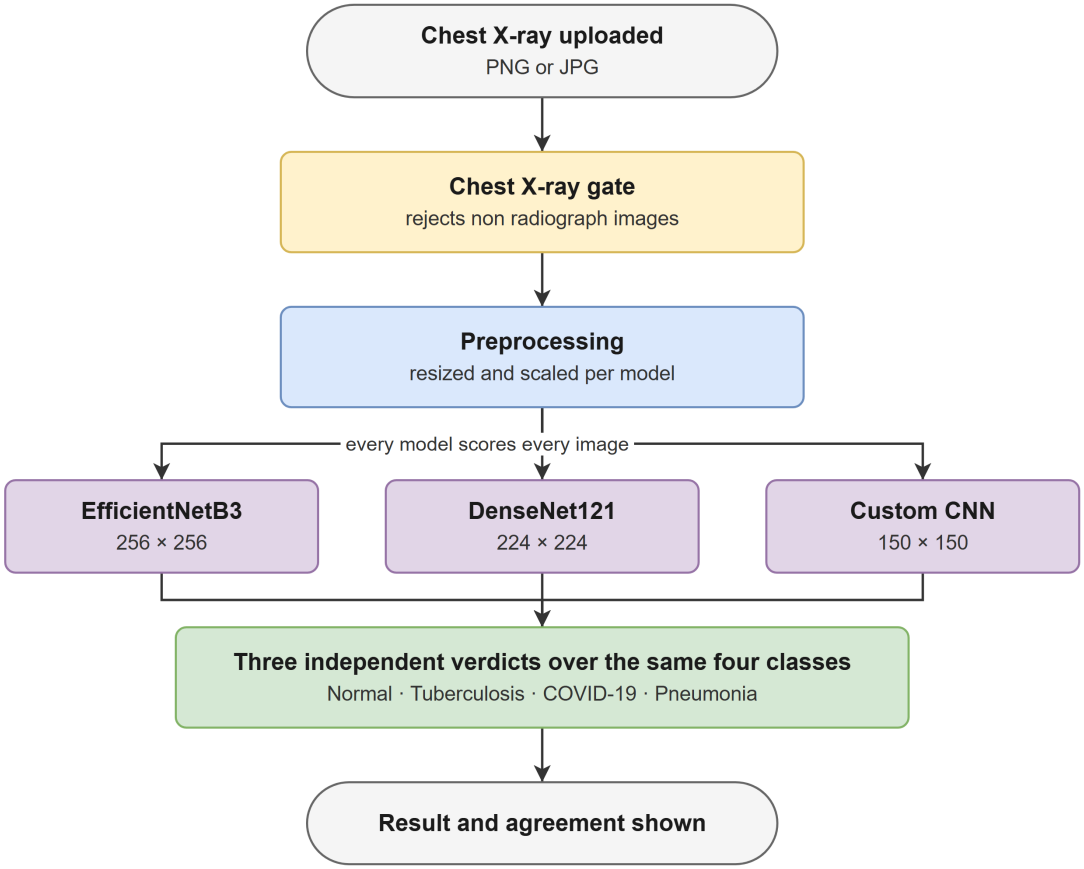


Figure 4.1: High-level inference flow for the unified four-class web application

Every trained four-class model scores every upload, always using the same label order: Normal, Tuberculosis, COVID-19, Pneumonia. Soft voting takes the average of the probability outputs. Old single-disease `.h5` files are rejected if they do not have four outputs, and if a weight file is missing the app reports it instead of crashing.

Table 4.1: Model integration overview

Architecture	Output	Input	Interface
EfficientNetB3	4-class softmax	256×256	Flask ensemble
DenseNet121	4-class softmax	224×224	Flask ensemble
Custom CNN	4-class softmax	150×150	Flask ensemble

4.2 Functional Requirements

1. Accept PNG/JPG/JPEG uploads (maximum 16 MB).
2. Check that the image is likely a chest X-ray; let the user override this, with a warning.
3. Run every available 4-class model on the same image.
4. Show the probability of each class for each model.
5. Calculate and show the soft-voting result and whether the models agree.
6. Show a clear error if weight files are missing or do not have four outputs.
7. Provide offline training scripts that can rebuild the weights from the manifest.

4.3 Non-Functional Requirements

Reproducibility: A fixed seed and a saved manifest, so results can be repeated.

Usability: One upload is enough; the user does not have to pick a model.

Robustness: Images that are not X-rays are rejected in several stages.

Consistency: The app processes images the same way as training.

Maintainability: The class order (`CLASSES`) is defined once in `training/common.py`.

Safety messaging: A clear note that the tool is for research only and not for clinical use.

4.4 Tools and Technologies Used

We used Python (3.9 or newer; 3.11 is recommended for recent TensorFlow), TensorFlow and Keras, OpenCV, Flask and Werkzeug, NumPy, scikit-learn (for evaluation), Matplotlib and Seaborn (for figures), HTML/CSS and Git. Training is best done on a machine with a good GPU. Prediction can run on a CPU, but it is slower. The main folders are `training/`, `Frontend/`, `weights/` and the three public dataset folders described in **Chapter 3**.

4.5 Chest X-Ray Validation Gate

1. **Monochrome or tint check:** the image must be grayscale or have a single colour tint (blue-tinted screens are allowed).
2. **Radiograph envelope:** the image's brightness, entropy, midtones, shape, symmetry and

similar measures must fall in the normal X-ray range; one measure is allowed to fail.

3. **Anatomy check:** the image is rejected only if there is no bright central column (the spine) and no left–right symmetry.

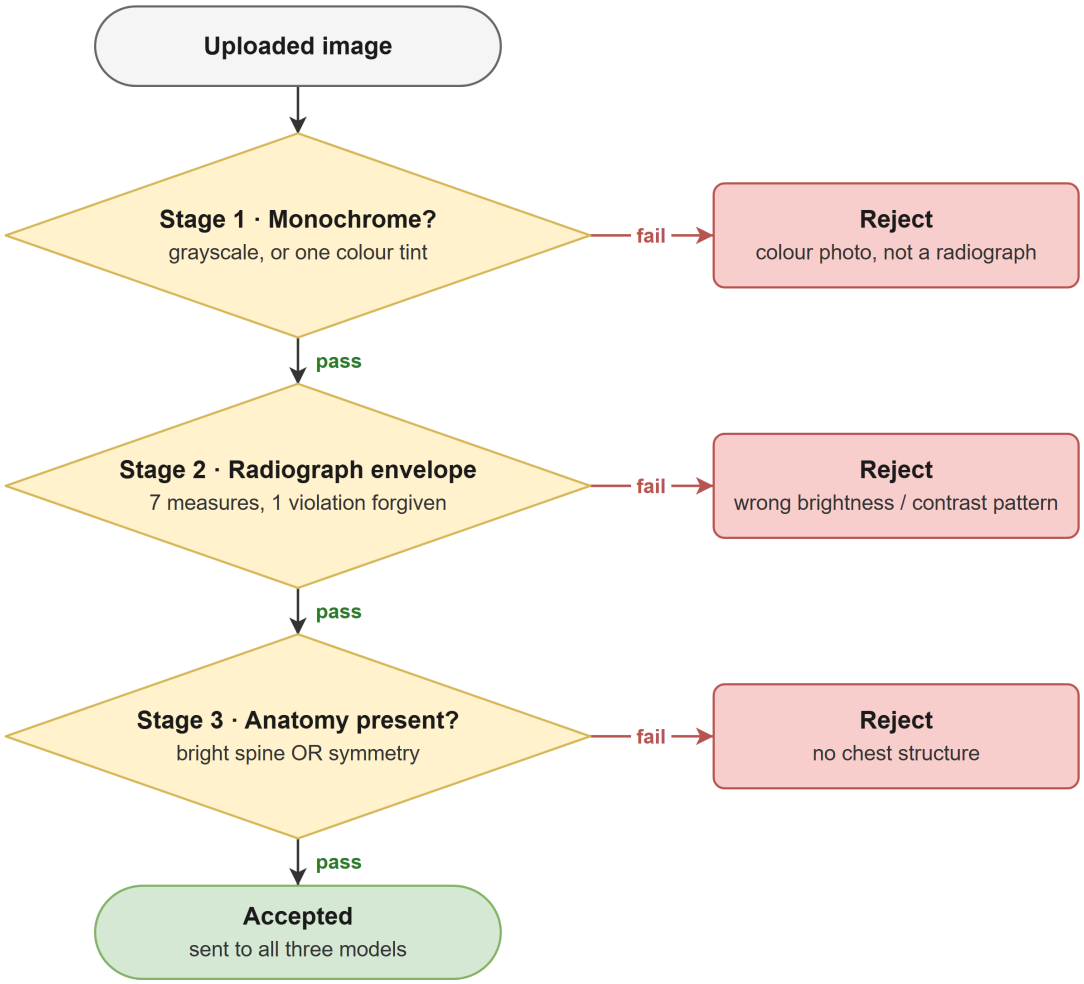


Figure 4.2: Three-stage chest X-ray validation gate with optional forced analysis

We tested the check on a labelled set of 351 real X-rays and 106 images that were not X-rays. It accepted all of the real X-rays (100%) and rejected about 77% of the non-X-rays, compared with about 21% for the old check that only looked at whether the image was monochrome. The user is told why an image was rejected, and the "Analyse anyway" option is protected against path traversal.

4.6 Application Flow and User Interface

The home page shows which models are loaded and ready. The user uploads an X-ray through a simple form. The result page shows the probability bars, a short note about each class,

the ensemble verdict, whether the models agree, and a warning if the user chose “Analyse anyway”. Figures 4.3 and 4.4 show the upload page and the rejection message. **Chapter 5** shows screenshots of the results for TB, COVID-19 and pneumonia examples.

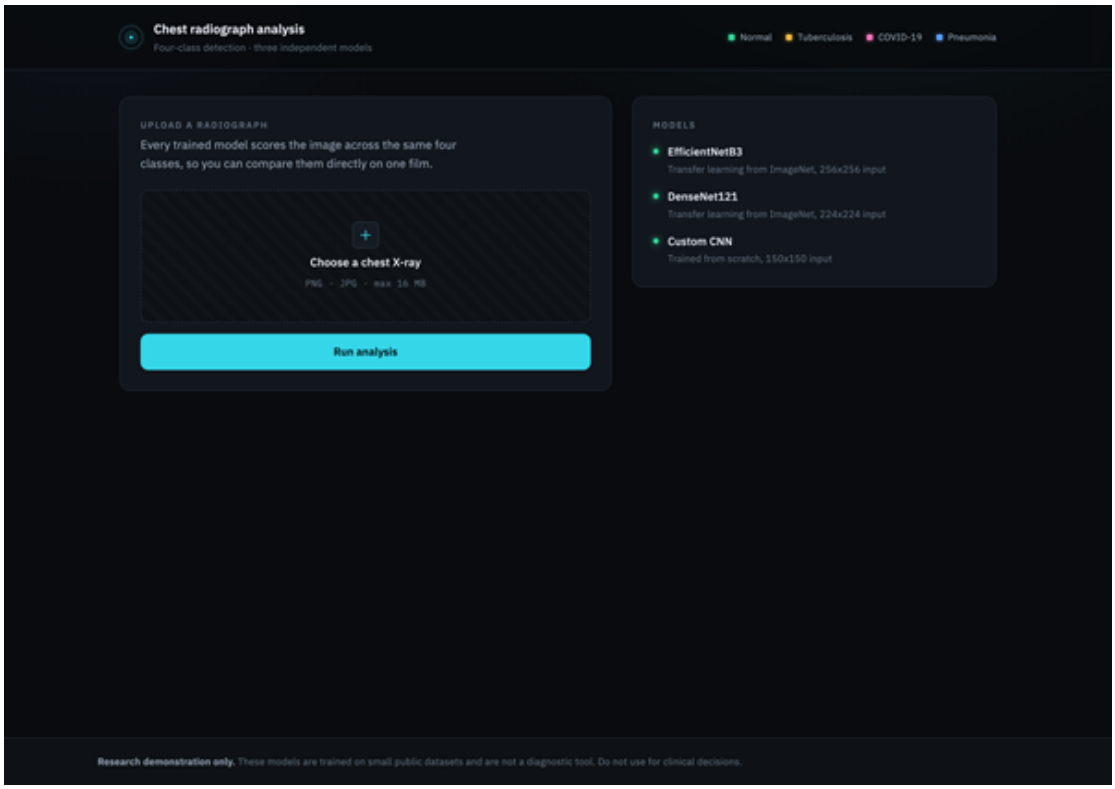


Figure 4.3: Flask upload interface for four-class multi-model chest radiograph analysis

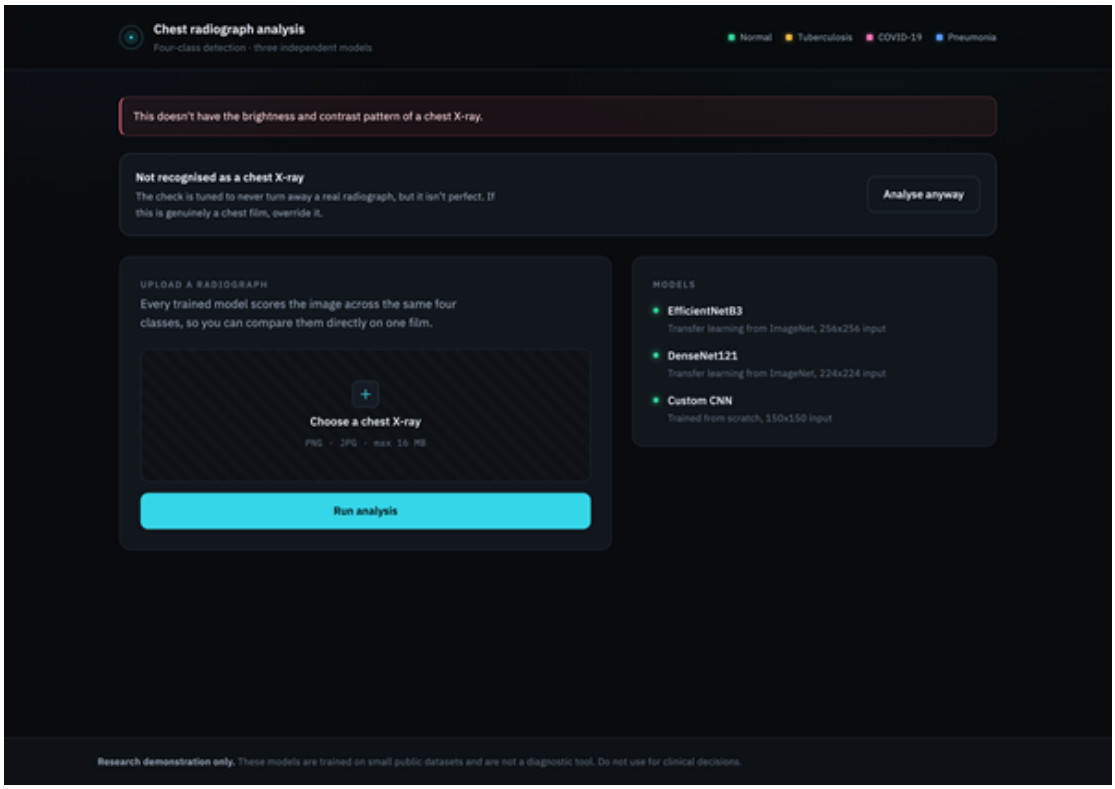


Figure 4.4: Chest X-ray validation gate rejection with Analyse anyway option

Table 4.2: Flask application components

Component	Location	Purpose
Web Framework	Flask	Routes and templates
Inference	`Frontend/predictors.py`	Gate, lazy load, ensemble
Shared constants	`training/common.py`	Class order, preprocess
Weights	`weights/*.keras`	4-class models

4.7 Train/Serve Consistency

The app reads images exactly as training does, with bilinear resizing and the same `preprocess_input` or division by 255. Training, evaluation and the user interface all use the same `CLASSES` order. Weight files that do not have four outputs are refused.

4.8 Summary

The system design turns the method into a research demo that people can actually use. It checks uploads more safely, always runs all the models, combines them with soft voting, and processes images in the app exactly as they were processed during training.

Chapter 5

Results and Discussion

This chapter gives the measured results of the four-class system on the held-out test set from `training/manifest.csv`. All accuracy, precision, recall, F1, confusion matrix and leakage probe numbers below come from the checked evaluation files (`training/results/comparison.md`, `results.json`, `ensemble.json`, `leakage_probe.json` and the run logs). Older single-disease scores are marked as prior work and are kept out of the four-class tables.

5.1 Experimental Setup

We used seed 1337, splits grouped by patient, and the class order Normal, Tuberculosis, COVID-19, Pneumonia. The final splits had 2,260, 485 and 482 images (70.0%, 15.0% and 14.9%). The models were trained with a batch size of 32 and with class weights based on how rare each class is (COVID-19 weight ≈ 2.851). In this run, each class was limited to at most 1,200 images (`--cap 1200`).

Table 5.1: Training environment (machine that produced the four-class weights)

Item	**Value**
OS	macOS 15.7.9 (x86_64)
CPU	Intel Core i9-9980HK @ 2.40 GHz (16 logical cores)
Memory	32 GB
GPU	None (CPU-only TensorFlow)
Python / TensorFlow / Keras	3.11.15 / 2.16.2 / 3.15.0
Training dates	DenseNet121: 4 Aug 2026; EfficientNetB3: 5 Aug 2026; CNN: 9 Aug 2026
Evaluation	Per-model comparison: 9 Aug 2026; ensemble + leakage: 13 Sep 2026

5.2 Unified Manifest Summary

After removing duplicate images (found with an MD5 hash), 10,328 images were left in the pool, with 45 duplicates removed. The COVID-19 class went down from 137 to **130** images. The final manifest has **3,227** images. Viral Pneumonia images from the COVID source were added to Pneumonia (89 `covid_db` images are labelled Pneumonia).

Table 5.2: Selected unified manifest counts by class and split

Class	Train	Val	Test	Total
Normal	840	180	180	1,200
Tuberculosis	488	105	104	697
COVID-19	91	20	19	130
Pneumonia	841	180	179	1,200
Total	**2,260**	**485**	**482**	**3,227**

Table 5.3: Class × source contingency (selected manifest)

Class	covid_db	pneumonia_db	tb_db	Total
Normal	88	556	556	1,200
Tuberculosis	0	0	697	697
COVID-19	130	0	0	130
Pneumonia	89	1,111	0	1,200
Total	**307**	**1,667**	**1,253**	**3,227**

Normal and Pneumonia come from more than one dataset on purpose. Tuberculosis and COVID-19 each come from only one dataset, so a model is more likely to recognise them by their source instead of the disease (see Section 5.6).

5.3 Four-Class Comparative Results

All models were tested on the same test set (n = 482). The soft-voting ensemble results come from the offline file `ensemble.json`, which averages the class probabilities of the three models.

Table 5.4: Overall four-class test metrics (n = 482)

Model	Accuracy	Bal. Acc.	Macro P	Macro R	Macro F1
EfficientNetB3	82.99%	83.00%	0.767	0.830	0.745
DenseNet121	90.87%	89.14%	0.850	0.891	0.864
Custom CNN	90.25%	88.36%	0.813	0.884	0.835
Ensemble (soft vote)	**92.32%**	**93.07%**	0.842	**0.931**	**0.867**

Of the single models, DenseNet121 does best on accuracy (90.87%), balanced accuracy (89.14%) and macro F1 (0.864). EfficientNetB3 has the highest COVID-19 recall of any single

model (89.47%), but its COVID precision is much lower. The ensemble raises overall accuracy to 92.32% and balanced accuracy to 93.07%.

Since there are only 19 COVID-19 test images, we treat **balanced accuracy** as the main metric rather than plain accuracy.

5.4 Per-Class Precision, Recall, and F1

5.4.1 Recall summary

Table 5.5: Per-class recall on the test set

Model	Normal	TB	COVID-19	Pneumonia
EfficientNetB3	78.3%	69.2%	89.5%	95.0%
DenseNet121	86.1%	88.5%	84.2%	97.8%
Custom CNN	80.6%	96.2%	78.9%	97.8%
Ensemble	85.6%	94.2%	94.7%	97.8%

5.4.2 DenseNet121 (best single-model macro F1)

Table 5.6: DenseNet121 per-class metrics

Class	Precision	Recall	F1	Support
Normal	92.3%	86.1%	0.891	180
Tuberculosis	97.9%	88.5%	0.929	104
COVID-19	59.3%	84.2%	0.696	19
Pneumonia	90.7%	97.8%	0.941	179

5.4.3 Ensemble (soft voting)

Table 5.7: Ensemble per-class metrics

Class	Precision	Recall	F1	Support
Normal	95.7%	85.6%	0.903	180
Tuberculosis	97.0%	94.2%	0.956	104
COVID-19	48.7%	94.7%	0.643	19
Pneumonia	95.6%	97.8%	0.967	179

COVID-19 precision is low for every model (no single model goes above about 60%). The ensemble reaches 94.7% COVID recall, but with only 19 test images ($n = 19$) this number can change a lot from sample to sample, so it should always be quoted with that warning.

5.5 Confusion Matrices

The classes are in the order Normal, Tuberculosis, COVID-19, Pneumonia. Rows show the true class and columns show the predicted class.

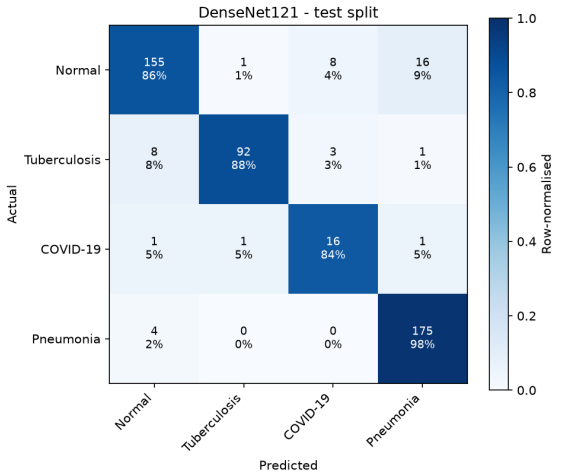


Figure 5.1: DenseNet121 confusion matrix

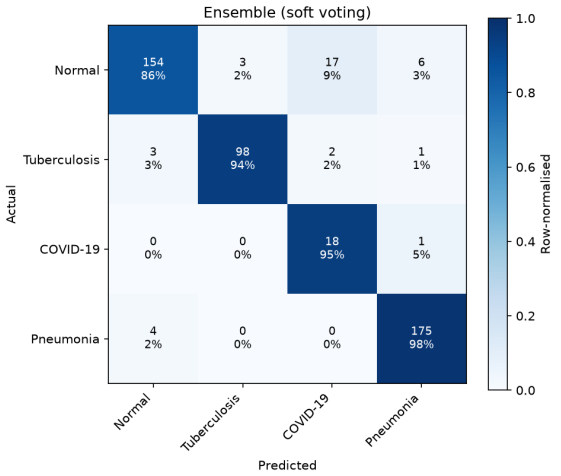


Figure 5.2: Ensemble confusion matrix

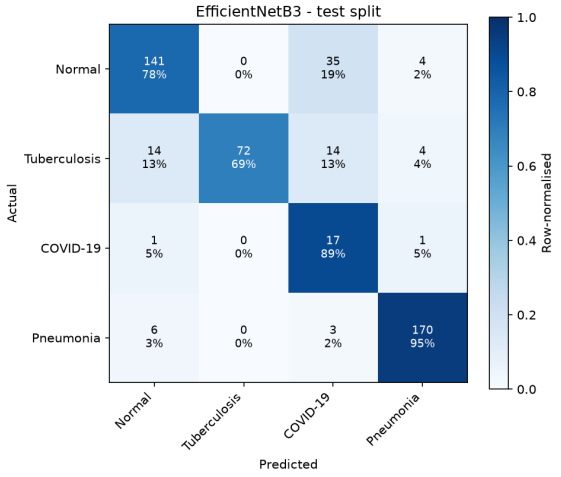


Figure 5.3: EfficientNetB3 confusion matrix

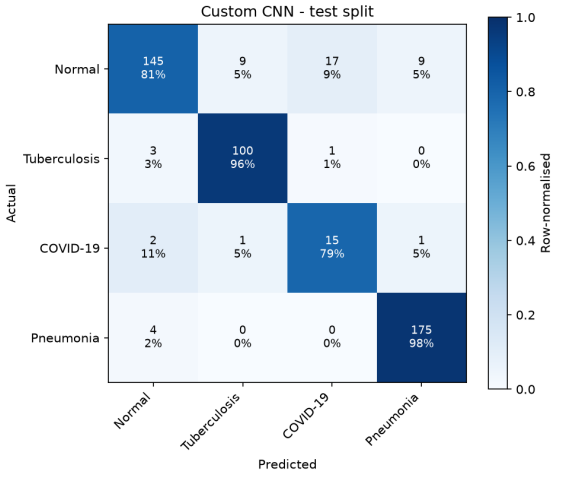


Figure 5.4: Custom CNN confusion matrix

5.6 Leakage Probe

We trained a probe (a custom CNN at 150×150, for 8 epochs) to guess which dataset each image came from. It was right **88.17%** of the time, compared with a majority-class baseline of **49.79%**, which is 38.4 percentage points higher. We rate this as **moderate**: the datasets can partly be told apart by source, but not easily or close to 100%. Since Tuberculosis and COVID-19 each come

from one source, part of their recall may come from scanner or processing differences. Normal and Pneumonia come from several sources, so they are less affected. Keep this in mind when reading the disease classification tables.

5.7 Training Run Summary

Table 5.8: Saved validation metrics and training cost

Model	Val Acc.	Val Loss	Epochs	Train time
EfficientNetB3	80.00%	0.492	20	~35.7 h (CPU)
DenseNet121	89.90%	0.299	20	310.6 min
Custom CNN	88.87%	0.354	20	36.8 min

The models have about 11.2 M (EfficientNetB3), 7.3 M (DenseNet121) and 1.25 M (Custom CNN) parameters. EfficientNetB3 took a long time to train because it was trained on the Mac's CPU only.

5.8 Deployed Interface Demonstrations

Figures 5.5–5.7 show the Flask app giving a combined soft-voting verdict, with probability bars for each model. The app uses the same soft-voting method as the offline ensemble results above.

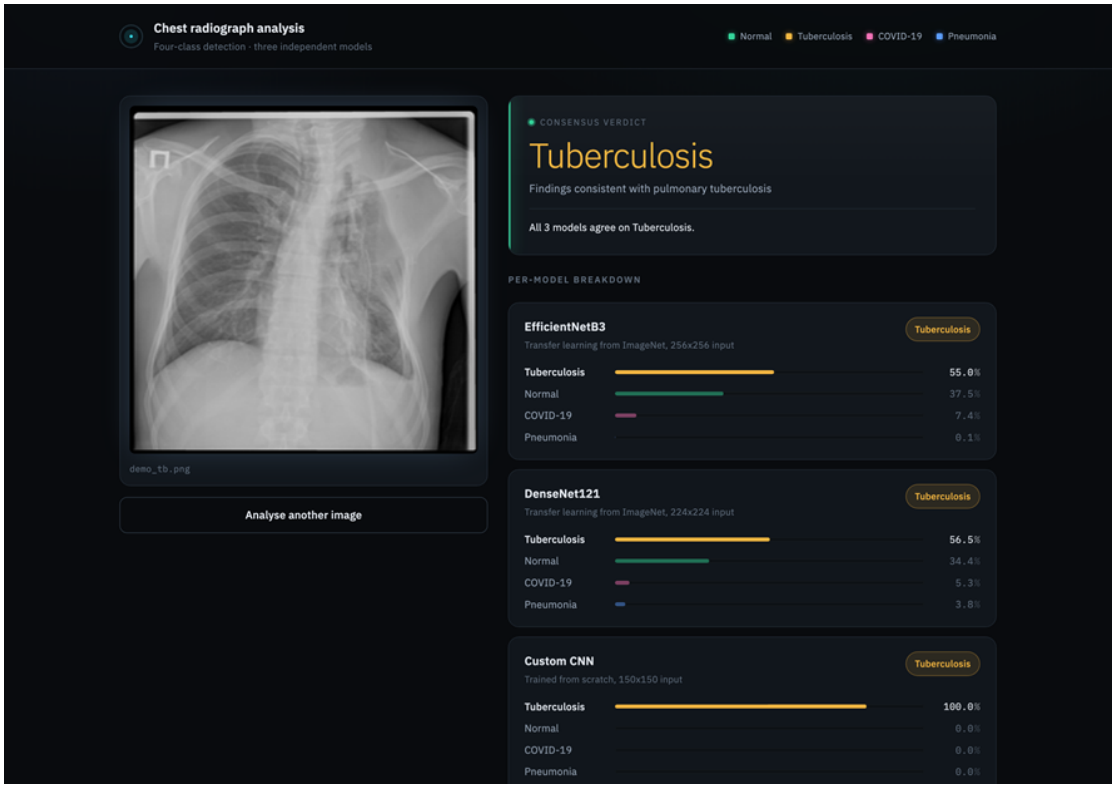


Figure 5.5: Consensus Tuberculosis verdict with per-model 4-class probability bars

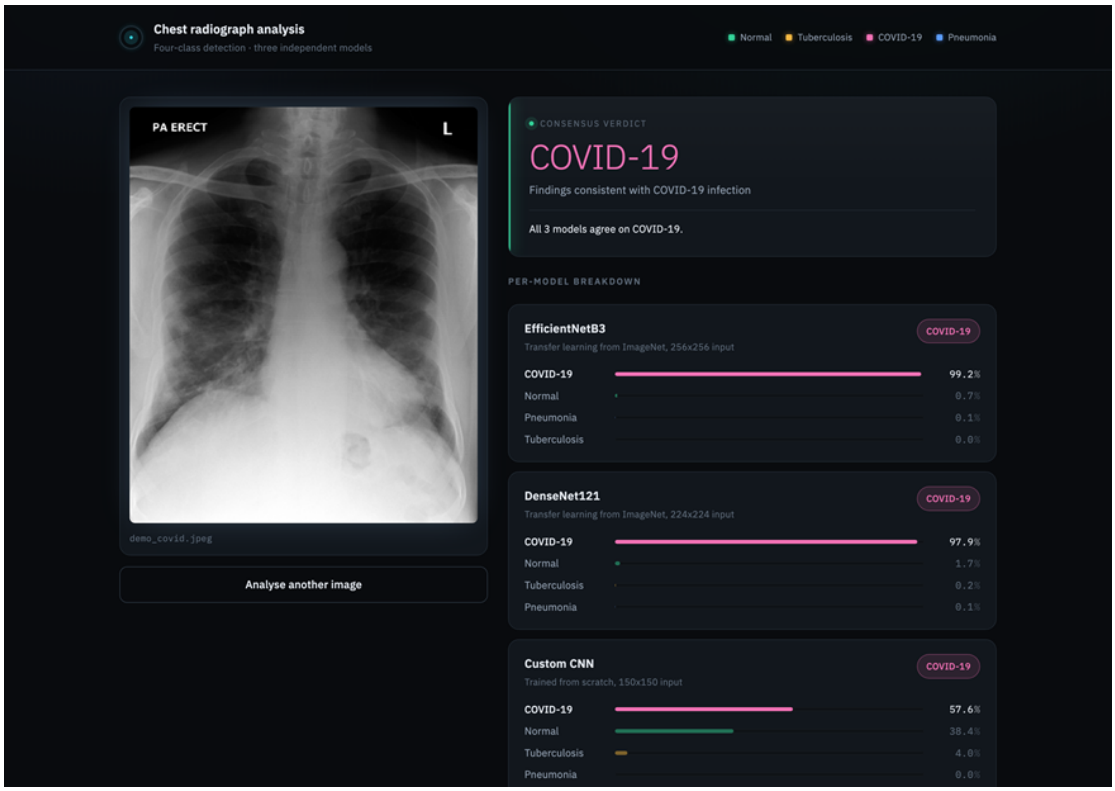


Figure 5.6: Consensus COVID-19 verdict with per-model 4-class probability bars

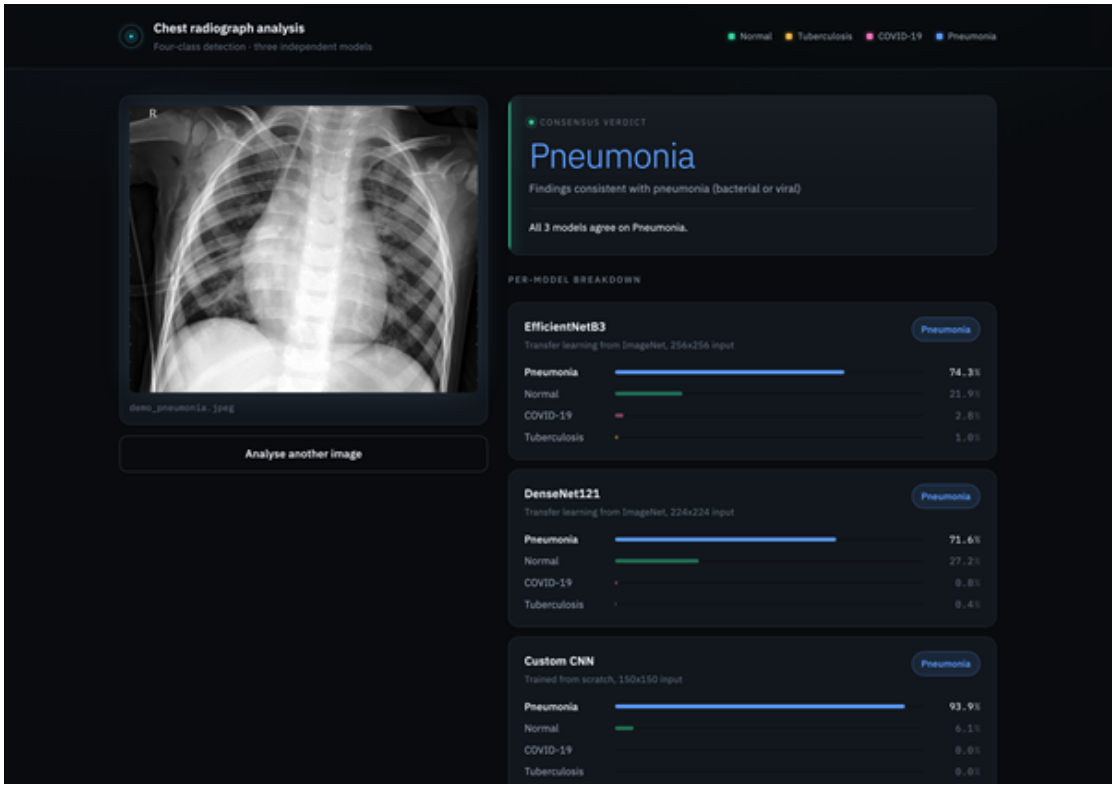


Figure 5.7: Consensus Pneumonia verdict with per-model 4-class probability bars

5.9 Challenges Faced and Solutions Applied

Table 5.9: Challenges and solutions during the 4-class conversion

#	Challenge	Solution
1	Old models could not be compared	One shared 4-class dataset for all models
2	Same patient or duplicate image in train and test	Split by patient and remove duplicate images
3	Wrong output layer (sigmoid) for 4 classes	Softmax output for all models
4	Models fine-tuned by different amounts	Unfreeze about 43% of each model
5	Non-X-ray images being accepted	New 3-step check (rejects 77% instead of 21%)
6	Images resized differently in training and in the app	Same resize method in both
7	Only accuracy was reported before	Added precision, recall and F1 (`evaluate.py`)
8	Very few COVID images (n=130)	Class weights; report COVID results separately

5.10 Historical Single-Disease Work

Our earlier two-class results (EfficientNetB3 TB test accuracy 95.63%, a pneumonia CNN at 92% on 624 images, and an old COVID model with validation close to 0% because of overfitting) **cannot be compared** with the four-class numbers in this chapter. They are only background from the earlier stage of the project.

5.11 Implementation Status

Table 5.10: Implementation status after evaluation export

Component	State
Unified manifest	Complete (3,227 images; Table 5.2)
Four-class weights	Present
Held-out evaluation	Complete (Tables 5.4–5.7)
Leakage probe	Complete (88.17% vs 49.79% baseline)
Frontend ensemble UI + gate	Complete

5.12 Summary of Findings

On the four-class test set (n = 482), DenseNet121 is the best single model (90.87% accuracy, 89.14% balanced accuracy, macro F1 0.864), and the soft-voting ensemble reaches 92.32% accuracy and 93.07% balanced accuracy. All models find pneumonia well, with recall of 95% or more. COVID-19 is still the hard class: recall can be high (94.7% for the ensemble) but precision stays modest, because there are only 19 COVID test images. The leakage probe shows a moderate source signal (88.17% against a 49.79% baseline), so the TB and COVID results, which each come from one source, should be read with care. Overall, the results meet the goal of the redesign: a fair comparison of several models on the same data, with metrics that show how the small classes perform instead of hiding them behind a single accuracy number.

Chapter 6

Future Recommendations

6.1 Conclusion

This project met its aim. We built a research chest X-ray system for several diseases that compares EfficientNetB3, DenseNet121 and a custom CNN on one shared four-class dataset (Normal, Tuberculosis, COVID-19, Pneumonia), and we made the models available through a Flask app that combines their results. On the held-out test set ($n = 482$), DenseNet121 was the best single model (90.87% accuracy, 89.14% balanced accuracy), and the soft-voting ensemble reached 92.32% accuracy and 93.07% balanced accuracy. The comparison can be trusted because of the shared manifest, patient-grouped splits, removal of duplicate images, the same fine-tuning share for each model, per-class metrics and the leakage probe (88.17% against a 49.79% baseline). The system is still a research demo built on public data and is not a clinical diagnostic device.

6.2 Recommendations

- Always report recall for each class, especially for small classes such as COVID-19.
- Keep splitting by patient whenever new data is added.
- Do not compare the old two-class results with the four-class results unless they are clearly marked as older work.
- Run the leakage probe whenever different public datasets are merged.

6.3 Future Enhancements

- More verified COVID data, and testing on data from several hospitals.
- Grad-CAM or similar heatmaps to show what the model is looking at.
- Support for DICOM files and a way to connect with hospital PACS systems.
- Calibrating the probabilities (for example with temperature scaling) and setting a separate decision threshold for each class.
- Using active learning or semi-supervised learning to grow the dataset.
- A mobile or cloud version with user login and audit logs.
- Adding more backbones (for example VGG19) using the same training setup.

6.4 Summary of Contributions

1. A shared four-class chest X-ray dataset built from three public sources, with controls against leakage.
2. A fair training setup for three architectures (the same head and the same share of fine-tuning).
3. Evaluation scripts that report precision, recall, F1, balanced accuracy, ensemble results and a leakage probe.
4. A Flask demo that runs all models, combines them with soft voting, shows whether they

agree, and checks uploads with a tested X-ray check.

5. Documentation that lets others repeat the work (manifest, scripts and status notes).

Bibliography

[1] World Health Organization. (2023). *Global Tuberculosis Report 2023*. Geneva: WHO.

[2] Deng, J., Dong, W., Socher, R., et al. (2009). ImageNet: A Large-Scale Hierarchical Image Database. *CVPR 2009*.

[3] Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv:1409.1556*.

[4] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*.

[5] Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. *CVPR 2017*.

[6] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*.

[7] Rajpurkar, P., Irvin, J., Ball, R. L., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv:1711.05225*.

[8] Wang, X., Peng, Y., Lu, L., et al. (2017). ChestX-ray8: Hospital-Scale Chest X-Ray Database and Benchmarks. *CVPR 2017*.

[9] Irvin, J., Rajpurkar, P., Ko, M., et al. (2019). CheXpert. *AAAI 2019*.

[10] Wang, L., Lin, Z. Q., & Wong, A. (2020). COVID-Net. *Scientific Reports*, 10(1), 19549.

[11] Chowdhury, M. E. H., Rahman, T., Khandakar, A., et al. (2020). Can AI Help in Screening Viral and COVID-19 Pneumonia? *IEEE Access*, 8, 132665–132676.

[12] Rahman, T., Khandakar, A., Qiblawey, Y., et al. (2021). Exploring Image Enhancement for COVID-19 Detection. *Computers in Biology and Medicine*, 132, 104319.

[13] Zhang, J., Xie, Y., Wu, Q., & Xia, Y. (2020). Medical Image Classification Using Synergic Deep Learning. *Medical Image Analysis*, 54, 10–19.

[14] Raghu, M., Zhang, C., Kleinberg, J., & Bengio, S. (2019). Transfusion: Understanding Transfer Learning for Medical Imaging. *NeurIPS 32*.

[15] World Health Organization. (2022). *Global Tuberculosis Report 2022*. Geneva: WHO.

[16] Jaeger, S., Candemir, S., Antani, S., et al. (2014). Two Public Chest X-Ray Datasets for Computer-Aided Screening of Pulmonary Diseases. *QIMS*, 4(6), 475–477.

[17] Rahman, T., et al. Tuberculosis (TB) Chest X-ray Database. Kaggle.

[18] Lakhani, P., & Sundaram, B. (2017). Deep Learning at Chest Radiography: Automated Classification of Pulmonary Tuberculosis. *Radiology*, 284(2), 574–582.

[19] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *NeurIPS 2012*.

[20] Hwang, S., Kim, H. E., Jeong, J., & Kim, H. J. (2016). A Novel Approach for Tuberculosis Screening Based on Deep CNNs. *SPIE Medical Imaging 2016*.

[21] Mooney, P. (2018). Chest X-Ray Images (Pneumonia). Kaggle.

[22] Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems.

[23] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

[24] Grinberg, M. (2018). *Flask Web Development*, 2nd ed. O'Reilly Media.

[25] Selvaraju, R. R., Cogswell, M., Das, A., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *ICCV 2017*.

Chapter A

Dataset Details

A.1 Dataset Overview

The combined dataset has four classes, and each image belongs to exactly one of them: Normal, Tuberculosis, COVID-19 or Pneumonia. In the reported run, COVID-19 has 130 images after removing duplicates. Normal and Pneumonia are limited to 1,200 images each, and class weights based on class size are used during training.

A.1.1 Data Sources

Table A.1: Dataset sources, paths, and known public links

Dataset	Local path	Supplies	Public link
TB Chest X-Ray	`TB_Chest_Radiography_Database/`	TB, Normal	Kaggle TB CXR corpus [17]
Covid19-dataset	`DenseNet121/Covid19-dataset/`	COVID-19, Normal, Pneumonia	Local COVID folder (317 images)
Chest X-Ray Pneumonia	`pneumonia_detection/data/chest_xray/`	Pneumonia, Normal	Kaggle CXR pneumonia corpus [21]

According to the checked evaluation files, the final manifest has 3,227 images: 2,260 for training, 485 for validation and 482 for testing. COVID-19 has 130 images after removing duplicates. Viral Pneumonia images from the COVID source are counted as Pneumonia. Tables of classes by source and by split are given in **Chapter 5**.

A.1.2 Preprocessing and Splits

Chapter 3 explains the patient-grouped train, validation and test splits (seed 1337), the removal of duplicate images, the shared geometric augmentation, and the input size and preprocessing used for each model.

Chapter B

Model Architecture Summaries

B.1 Model Training and Implementation

- EfficientNetB3 / DenseNet121: ImageNet base plus shared head (GAP, Dropout, Dense 256, Dropout, Softmax 4); Phase 1 frozen; Phase 2 unfreeze from layers 340 / 300 (~43%).
- Custom CNN: five convolution blocks ending in Softmax(4).
- Scripts: `training/build_manifest.py`, `common.py`, `models.py`, `train.py`, `evaluate.py`, `leakage_probe.py`; frontend under `Frontend/`; weights `efficientnetb3_4class.keras`, `densenet121_4class.keras`, `cnn_4class.keras`.
- Run order: build the manifest, train all models, evaluate, optionally run the leakage probe, and then start `Frontend/app.py`. The portable bundle in `xray-classifier-portable/` already includes the three `*.keras` weight files, so the demo can be run without retraining on your own machine.

Chapter C

Alignment with Sustainable Development Goals (SDGs)

Table C.1: Potential educational and research alignment with SDGs (non-clinical)

No.	SDG	How this project relates
1	SDG 3 Good Health and Well-being	Research towards easier screening of lung diseases; not a diagnostic tool.
2	SDG 4 Quality Education	Can be used to teach how to build and test medical AI fairly.
3	SDG 9 Industry, Innovation and Infrastructure	Open software for comparing several X-ray models.
4	SDG 10 Reduced Inequalities	Could help areas with few radiologists in future, after clinical testing.